Backend & System Architecture Roadmap

For an Angular developer who knows basic Node.js CRUD and basic MongoDB queries. Part 1 is a hands-on build roadmap; Part 2 is an interview preparation track you run alongside it.

First, a reframe

Three years of Angular is not wasted. You understand state, components, async flows, and how APIs feel from the consumer side, and that is real skill. What limits growth is seeing only one half of the system. Senior and architect roles need you to reason about the whole path of a request, from the Angular click to the database and back. This roadmap fixes exactly that.

How we will learn: one project, many layers

The most common mistake is building ten separate toy apps. You learn syntax but never learn *when to use what*, because you never feel the problem the tool solves. Instead, you will build one project that grows, and in every phase follow the same loop:

1. **Build the naive version.**
2. **Break it or measure it.** Load test it, fire concurrent requests at it, kill a process.
3. **Introduce the tool that fixes the pain.**

> **The capstone: TicketRush, an event ticket booking platform** (like BookMyShow or Ticketmaster). It naturally needs every concept: 1,000 people booking the last seat forces transactions and locking; event listings are read constantly (caching); seat maps update live (real-time); ticket PDFs and emails shouldn't block the user (queues); payments and user data (security); flash sales (scaling).
>
> **Stack:** Angular frontend (your strength). **Express on Node.js** for the backend in Phases 1–8, with the layered structure imposed by you. In Phase 9 you build one extracted service in **NestJS**, so you learn both and understand exactly what a framework automates.

Progress tracker

At roughly 10–12 hours a week, Phases 0–8 take about 5–6 months. The interview track runs in parallel.

Phase	Topic	Time	Started	Done
0	How the web actually works	1–2 weeks	☐	☐
1	A real backend structure with Express	3 weeks	☐	☐
2	Databases for real	3–4 weeks	☐	☐
3	Caching and Redis	2 weeks	☐	☐
4	Real-time	1–2 weeks	☐	☐
5	Async processing and message queues	3 weeks	☐	☐
6	Security	2 weeks, then forever	☐	☐
7	Docker, deployment, and CI/CD	2 weeks	☐	☐
8	Observability and load	2 weeks	☐	☐
9	Architecture patterns, NestJS, system design	Ongoing	☐	☐
A–M	Interview preparation track (Part 2)	Parallel, from week 1	☐	☐

Part 1: Hands-on roadmap

Each phase adds a layer to TicketRush. Don’t move on until you’ve built it, broken it, and written a short decision log entry.

> **How to read the resource lists.** Every resource is a clickable link, and each list is ordered best first: begin with the item marked **START HERE** and only go further down if you want more depth. ★ numbers are GitHub stars, checked directly on GitHub on 18 October 2026, which is a good signal of how strongly the developer community recommends a repo. Links to talks open a YouTube search so they keep working even if a video is re-uploaded. The appendix lists every GitHub repo in this roadmap ranked by stars.

PHASE 0

How the web actually works

1–2 weeks

Learn

- HTTP in depth: methods, status codes, headers, cookies, caching headers.
- Networking basics: DNS, TCP vs UDP, TLS/HTTPS.
- What a reverse proxy is, and basic Linux terminal usage.

Do

Call APIs with `curl`, read every header in the DevTools Network tab with fresh eyes, and set up the TicketRush repo with a README listing these phases.

Resources · best first

- **START HERE** MDN Web Docs: HTTP · start with the “Overview” guide; the reference every web developer uses
  developer.mozilla.org/en-US/docs/Web/HTTP
- Hussein Nasser (YouTube) · the best backend-fundamentals channel: HTTP, TCP, proxies
  youtube.com/@hnasr
- vasanthk/how-web-works ★ 17k · what happens behind the scenes of a web request
  github.com/vasanthk/how-web-works
- roadmap.sh Backend roadmap ★ 370k · a map of the territory; star count is its GitHub repo
  roadmap.sh/backend

☐ Phase complete ☐ Decision log written ☐ Results measured

PHASE 1

A real backend structure with Express

3 weeks

Why Express here

You already know it, so you’ll move fast. The catch: Express gives you total freedom, which is how people end up with 800-line route files mixing validation, business logic, and queries. In this phase you impose the structure yourself, which teaches you *why* layers exist.

Folder structure to follow

```
src/
  modules/
    bookings/
      booking.routes.ts       // HTTP only: parse request, call service
      booking.service.ts      // business rules: "can this seat be booked?"
      booking.repository.ts   // database queries only
      booking.schema.ts       // validation (zod)
  middlewares/                // auth, error handler, rate limiter
  config/                     // environment variables
  app.ts                      // wires everything together
```

The one rule

Routes never touch the database, and repositories never contain business logic. Services depend on repositories; routes depend on services.

Learn

- Request validation with zod, a central error-handling middleware, environment config.
- API design: REST conventions, proper status codes, pagination, versioning.
- Auth: password hashing (bcrypt/argon2), JWT access and refresh tokens, sessions vs JWT, role-based access (admin vs user) as middleware.
- TypeScript in the backend, and basic tests with Supertest from day one.

Build in TicketRush

Users sign up, admins create events, venues, and seats, and users browse events.

Resources · best first

- **START HERE** goldbergyoni/nodebestpractices ★ 106k · sections 1 (project architecture) and 2 (error handling)
  github.com/goldbergyoni/nodebestpractices
- Express docs · routing, middleware, and error-handling guides
  expressjs.com/en/guide/routing.html
- Zod docs · schema validation for request bodies
  zod.dev
- Sairyss/backend-best-practices ★ 2.4k · backend practices with TypeScript examples
  github.com/Sairyss/backend-best-practices

☐ Phase complete ☐ Decision log written ☐ Results measured

PHASE 2
Databases for real

3–4 weeks · most important

Learn

- PostgreSQL and SQL: joins and aggregations; schema design, normalization, migrations.
- Indexes and reading `EXPLAIN ANALYZE`.
- Transactions, ACID, isolation levels, row locking (`SELECT ... FOR UPDATE`).
- The N+1 query problem and connection pooling.
- Replication, read replicas, and sharding at a conceptual level only.

Driver choice

Use the plain pg driver and write raw SQL first, so you actually learn SQL. Add an ORM (Prisma or Drizzle) only once you're comfortable, and compare the queries it generates.

Build in TicketRush

The booking flow, inside a transaction. Keep MongoDB for event reviews and the activity feed, so you experience where each database fits.

Break it

Write a script that fires 100 concurrent requests to book the same seat. Without proper locking you will double-book. Fix it with a row lock or a unique constraint. This single experiment teaches more than ten videos.

Measure it

Seed 1 million bookings, time a query, add an index, and compare the `EXPLAIN` output before and after.

Resources · best first

- **START HERE** Designing Data-Intensive Applications (book) · Martin Kleppmann; chapters 2, 3, and 7 for this phase
  dataintensive.net
- Use The Index, Luke · free, and the best explanation of SQL indexing
  use-the-index-luke.com
- Hussein Nasser: database videos · transactions, isolation levels, indexing
  YouTube search: hussein nasser database engineering
- node-postgres (pg) docs · the driver you'll write raw SQL with
  node-postgres.com
- pingcap/awesome-database-learning ★ 11k · go deeper into database internals
  github.com/pingcap/awesome-database-learning
- MongoDB University · free official courses
  learn.mongodb.com
- Arpit Bhayani · database internals and system design deep dives
  arpitbhayani.me

☐ Phase complete ☐ Decision log written ☐ Results measured

PHASE 3
Caching and Redis

2 weeks

Learn

- The cache-aside pattern, TTLs, and cache invalidation (the genuinely hard part).
- Cache stampede, and what *not* to cache.
- Redis data structures: strings, hashes, sets, sorted sets, pub/sub.
- Distributed locks with `SET NX PX`.

Build in TicketRush

- Cache the event listings.
- A rate limiter on login and booking: first a fixed window, then a sliding window.
- Seat holds: when a user selects a seat, hold it for 10 minutes using a Redis key with a TTL. This is exactly what real ticketing sites do.
- A "trending events" list with sorted sets.

Measure it

Load test `/events` with k6 before and after caching, and compare p95 latency.

Resources · best first

- **START HERE** Redis docs: data types · strings, hashes, sets, sorted sets, TTLs
  redis.io/docs/latest/develop/data-types
- Redis University · free official courses
  university.redis.io
- ByteByteGo: caching strategies · short visual explanations of cache patterns
  YouTube search: bytebytego caching strategies

☐ Phase complete ☐ Decision log written ☐ Results measured

PHASE 4
Real-time
1–2 weeks

Learn

Polling, long polling, Server-Sent Events (SSE), and WebSockets, and when each fits. Also learn why WebSockets are stateful and therefore hard to scale.

Build in TicketRush

A live seat map: when anyone holds or books a seat, every viewer sees it turn grey instantly (Angular + Socket.IO on your Express server). Build notifications with SSE as a comparison.

Break it

Run two backend instances behind Nginx. Users connected to instance 1 won't see updates from instance 2. Fix it with the Socket.IO Redis adapter and sticky sessions. Now you truly understand horizontal scaling.

Resources · best first

- **START HERE** Socket.IO docs: Using multiple nodes · sticky sessions and the Redis adapter
  socket.io/docs/v4/using-multiple-nodes
- Hussein Nasser: WebSockets vs SSE · trade-offs of each real-time option
  YouTube search: hussein nasser websockets server sent events

☐ Phase complete ☐ Decision log written ☐ Results measured

PHASE 5
Async processing and message queues
3 weeks

Learn

- Why async matters: users shouldn't wait for an email to send.
- Job queues vs message brokers vs event streams.
- Retries with backoff and dead-letter queues.
- At-least-once delivery, idempotency, and the outbox pattern.

Build in TicketRush, in three steps

- **BullMQ (on Redis):** after a booking, a worker process generates the PDF ticket and sends the email. Kill the worker mid-job and watch the retry happen.
- **RabbitMQ:** the booking service publishes a `booking.confirmed` event, and separate email, analytics, and loyalty-points consumers each react to it (fan-out).
- **Kafka (optional):** stream "seat viewed" and "seat booked" events for analytics, and practice replaying them.

Break it

Deliver the same message twice, then make the consumer idempotent so the user doesn't get two emails.

Resources · best first

- **START HERE** RabbitMQ official tutorials · do tutorials 1–5 in JavaScript
  rabbitmq.com/tutorials
- rabbitmq/rabbitmq-tutorials ★ 6.9k · runnable code for those tutorials
  github.com/rabbitmq/rabbitmq-tutorials
- BullMQ docs · job queues on Redis for Node
  docs.bullmq.io
- CodeOpinion (YouTube) · event-driven architecture, outbox, idempotency
  youtube.com/@CodeOpinion
- Confluent Developer · free Kafka courses
  developer.confluent.io

☐ Phase complete ☐ Decision log written ☐ Results measured

PHASE 6
Security

2 weeks, then forever

Learn

- The OWASP Top 10.
- SQL and NoSQL injection. Mongo's $ne injection is directly relevant to you.
- XSS and CSRF.
- CORS from the *server* side (you've fought it from Angular; now you'll understand it).
- Broken access control, especially IDOR: can user A see user B's booking just by changing the ID in the URL?
- JWT pitfalls, the helmet middleware, and secrets management.

Build in TicketRush

Audit the app against the OWASP Top 10 and fix what you find. Add refresh-token rotation.

Practice

Hack OWASP Juice Shop, a deliberately vulnerable app built with Angular and Node, so you'll recognize the stack.

Resources · best first

- **START HERE** PortSwigger Web Security Academy · free hands-on labs; outstanding
  portswigger.net/web-security
- OWASP Top 10 · the list every interviewer expects you to know
  owasp.org/www-project-top-ten
- OWASP Cheat Sheet Series ★ 34k · concise fixes for each vulnerability; star count is its GitHub repo
  cheatsheetseries.owasp.org
- juice-shop/juice-shop ★ 14k · deliberately vulnerable Angular + Node app to hack
  github.com/juice-shop/juice-shop

☐ Phase complete ☐ Decision log written ☐ Results measured

PHASE 7

Docker, deployment, and CI/CD

2 weeks

Learn

- Docker images and containers, and docker-compose.
- Nginx as a reverse proxy and load balancer.
- 12-factor configuration and graceful shutdown on SIGTERM.
- GitHub Actions for CI: lint, test, build.
- Cloud basics at the concept level: AWS EC2, S3, RDS.

Build in TicketRush

Make the whole system run with one `docker compose up`: Angular, two API instances, the worker, Postgres, Mongo, Redis, and RabbitMQ. Deploy it to a cheap VPS.

Resources · best first

- **START HERE** Docker docs: Get started · the official walkthrough
  docs.docker.com/get-started
- TechWorld with Nana (YouTube) · Docker, Kubernetes, and DevOps explained clearly
  youtube.com/@TechWorldwithNana
- prakhar1989/docker-curriculum ★ 6.1k · hands-on Docker tutorial
  github.com/prakhar1989/docker-curriculum
- The Twelve-Factor App · configuration and deployment principles
  12factor.net
- milanm/DevOps-Roadmap ★ 21k · optional: where DevOps goes next
  github.com/milanm/DevOps-Roadmap

☐ Phase complete ☐ Decision log written ☐ Results measured

PHASE 8

Observability and load

2 weeks

Learn

- Structured logging with pino, and correlation IDs.
- Metrics using RED: rate, errors, duration.
- Latency percentiles: p50, p95, p99.
- Health checks and tracing with OpenTelemetry.

Build in TicketRush

A Prometheus + Grafana dashboard. Then simulate a flash sale with k6 (thousands of virtual users booking at once) and find the bottleneck. It’s often the database connection pool. This is the moment everything connects.

Resources · best first

- **START HERE** Grafana k6 docs ★ 32k · load testing; star count is the grafana/k6 repo
  grafana.com/docs/k6/latest
- Prometheus docs · metrics collection
  prometheus.io/docs/introduction/overview
- OpenTelemetry for JavaScript · tracing
  opentelemetry.io/docs/languages/js
- Pino docs · fast structured logging for Node
  getpino.io

☐ Phase complete ☐ Decision log written ☐ Results measured

PHASE 9

Architecture patterns, NestJS, system design

Ongoing

Learn

- Monolith → modular monolith → microservices, and why starting with microservices is usually a mistake.
- API gateways; sync (REST/gRPC) vs async (events) communication between services.
- Saga pattern and circuit breakers.
- CAP theorem and consistency models.
- Load balancing, CDNs, and back-of-envelope estimation.

Build in TicketRush, with NestJS

Extract notifications into a separate service that talks to the rest of the system only via RabbitMQ, and build this new service in **NestJS**. Its modules, providers, and dependency injection will feel like Angular, and after building layers by hand in Express you'll see exactly what it automates. Feel the cost of distributed debugging and eventual consistency, then write down whether the split was worth it.

Practice

Once a week, design a classic system on paper, then compare with published solutions (see Track H in Part 2).

Resources · best first

- **START HERE** donnemartin/system-design-primer ★ 375k · the most-starred system design resource on GitHub
  github.com/donnemartin/system-design-primer
- ByteByteGoHq/system-design-101 ★ 91k · complex systems explained with visuals
  github.com/ByteByteGoHq/system-design-101
- binhnguyennus/awesome-scalability ★ 75k · real engineering blogs from large companies
  github.com/binhnguyennus/awesome-scalability
- karanpratapsingh/system-design ★ 47k · a free, complete system design course
  github.com/karanpratapsingh/system-design
- NestJS docs: RabbitMQ microservices · for your extracted notification service
  docs.nestjs.com/microservices/rabbitmq
- Sairyss/domain-driven-hexagon ★ 15k · architecture patterns with TypeScript code
  github.com/Sairyss/domain-driven-hexagon
- Martin Fowler: Microservices · read with "MonolithFirst" (martinfowler.com/bliki/MonolithFirst.html)
  martinfowler.com/articles/microservices.html
- ByteByteGo (YouTube) · short system design explainers
  youtube.com/@ByteByteGo
- Gaurav Sen (YouTube) · system design walkthroughs
  youtube.com/@gkcs

☐ Phase complete ☐ Decision log written ☐ Results measured

“When to use what” cheat sheet

Situation	Reach for	Avoid when
Money, bookings, relational data	**PostgreSQL with transactions**	Rarely the wrong default
Flexible, document-shaped data (reviews, logs, catalogs)	**MongoDB**	You need multi-entity transactions or heavy joins
Data read often, changed rarely	**Redis cache**	Data must always be exactly fresh (e.g., balances)
Temporary state (seat holds, OTPs, sessions)	**Redis with TTL**	The data must survive permanently
Limiting abuse	**Redis rate limiter**	Never; always protect login and expensive endpoints
Two-way, low-latency updates (chat, live seat maps)	**WebSockets**	Updates only flow server → client
Server → client only (notifications, progress, streaming)	**SSE**	You need client → server messages on the same channel
Updates every few minutes are fine	**Polling**	Real-time matters
Slow work the user shouldn’t wait for (emails, PDFs, images)	**Job queue (BullMQ)**	The user needs the result immediately
One event, many independent reactions	**Message broker (RabbitMQ)**	It’s a simple single background job
Huge event volume, replay, analytics	**Kafka**	Low volume; it’s operationally heavy
CPU-heavy work in Node (image processing, reports)	**Worker threads or a separate worker via a queue**	The work is I/O-bound; async I/O already handles it
Small app, small team, fast iteration	**Express**	Many developers need enforced conventions
Larger codebase, many developers, Angular-style structure	**NestJS**	A tiny service where the framework is overhead
Splitting the system up	**Microservices**	No team, deployment, or scaling reason exists yet

Part 2: Interview preparation track

Run this alongside Part 1 from week 1. The roadmap gives you real experience; this track makes sure you can *explain* it under interview pressure, and covers topics that interviews ask about but a single project won't naturally teach. Backend and full-stack interviews typically include some mix of: a Node/JavaScript deep-dive, a database and API discussion, a low-level design round, a system design round, a DSA round, and a behavioral round.

TRACK A

Node.js internals

Asked in almost every Node interview

Know

- Event loop phases (timers, poll, check) and the microtask queue.
- Execution order of `process.nextTick`, `Promise.then`, `setTimeout(0)`, and `setImmediate`.
- The libuv thread pool: what uses it (fs, crypto, dns.lookup, zlib) and why its default size of 4 matters.
- Why CPU-heavy work blocks every request, and the fixes: worker threads, or offloading to a queue.
- Streams, piping, and backpressure; when to stream instead of loading a whole file into memory.
- Using all CPU cores: the cluster module and PM2.
- Memory leaks, heap snapshots, and garbage collection basics.
- Error handling: async errors, `unhandledRejection`, and graceful shutdown.

Resources · best first

- **START HERE** Node.js docs: The Event Loop · the official explanation of phases and nextTick
  nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick
- Node.js docs: Don't Block the Event Loop · the thread pool and CPU-heavy work
  nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop
- Philip Roberts: "What the heck is the event loop anyway?" · the classic JSConf talk
  YouTube search: philip roberts what the heck is the event loop
- Node.js Design Patterns (book) · Casciaro and Mammino; streams, async patterns, scaling
  nodejsdesignpatterns.com

TRACK B

JavaScript and TypeScript depth

Foundation for every coding round

Know

- Closures, `this`, prototypes, and scope.
- Promises in depth: `Promise.all` vs `allSettled` vs `race` vs `any`; error propagation in async/await.
- TypeScript generics, utility types, type narrowing, and discriminated unions.
- CommonJS vs ES modules.

Resources · best first

- **START HERE** getify/You-Dont-Know-JS ★ 185k · free book series on JS in depth
  github.com/getify/You-Dont-Know-JS
- TypeScript Handbook · the official guide
  typescriptlang.org/docs/handbook/intro.html
- sudheerj/javascript-interview-questions ★ 28k · quick revision before interviews
  github.com/sudheerj/javascript-interview-questions

TRACK C

API design beyond CRUD

Shows senior-level thinking

Know

- REST vs GraphQL vs gRPC, and when each fits.
- Offset vs cursor pagination, and why cursor pagination scales better.
- Idempotency keys: how to make a “create payment” endpoint safe to retry.
- Consistent error formats, status codes, and API versioning.
- OpenAPI/Swagger documentation.
- Webhooks: sending them reliably, and verifying signatures when receiving them.

Resources · best first

- **START HERE** microsoft/api-guidelines ★ 23k · REST API design guidelines
  github.com/microsoft/api-guidelines
- Stripe docs: idempotent requests · a model of excellent API design; also read their pagination and webhook docs
  docs.stripe.com/api/idempotent_requests
- GraphQL: Learn · official introduction
  graphql.org/learn

TRACK D

Authentication and authorization in depth

Very common for full-stack roles

Know

- OAuth 2.0 authorization code flow with PKCE, step by step.
- OpenID Connect, “Login with Google,” and single sign-on (SSO) at the concept level.
- Cookie flags: HttpOnly, Secure, SameSite, and how they relate to XSS and CSRF.
- Where to store tokens in an Angular app, and the trade-offs of each option.
- RBAC vs ABAC (role-based vs attribute-based access control).

Resources · best first

- **START HERE** Nate Barbettini: “OAuth 2.0 and OpenID Connect (in plain English)” · the clearest OAuth talk
  YouTube search: nate barbettini oauth 2.0 and openid connect in plain english
- oauth.net: OAuth 2.0 · flows, PKCE, and specs
  oauth.net/2
- OWASP Authentication Cheat Sheet · also read the Session Management cheat sheet
  cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

TRACK E

Testing

Expected at mid and senior level

Know

- The test pyramid: unit, integration, end-to-end.
- Unit tests with Jest or Vitest; API tests with Supertest.
- Integration tests against a real database with Testcontainers.
- When to mock and when not to; testing queue consumers and idempotency.

Build in TicketRush

Start in Phase 1. The most valuable test you'll write: one proving that two concurrent bookings for the same seat can't both succeed.

Resources · best first

- **START HERE** goldbergyoni/javascript-testing-best-practices ★ 25k · the reference for JS/Node testing
  github.com/goldbergyoni/javascript-testing-best-practices
- goldbergyoni/nodejs-testing-best-practices ★ 4.4k · beyond the basics: integration and component tests
  github.com/goldbergyoni/nodejs-testing-best-practices
- Vitest docs · fast test runner
  vitest.dev
- forwardemail/supertest ★ 14k · HTTP API testing
  github.com/forwardemail/supertest
- testcontainers/testcontainers-node ★ 2.6k · tests against a real Postgres or Redis
  github.com/testcontainers/testcontainers-node

TRACK F

Distributed systems building blocks

The vocabulary of system design rounds

Know

- CAP theorem and PACELC, with real examples.
- Leader-follower replication, replication lag, and read-your-writes consistency.
- Sharding strategies and consistent hashing.
- Rate limiting algorithms: token bucket, leaky bucket, fixed window, sliding window.
- Unique ID generation: auto-increment vs UUID vs Snowflake-style IDs.
- Bloom filters, and what problems they solve.
- Distributed locks and their pitfalls; sagas vs two-phase commit at the concept level.
- At-most-once, at-least-once, and "exactly-once" delivery.

Resources · best first

- **START HERE** Designing Data-Intensive Applications (book) · chapters 5, 6, 8, and 9
  dataintensive.net
- Martin Kleppmann: Distributed Systems lectures · University of Cambridge lecture series
  YouTube search: martin kleppmann distributed systems lecture
- ByteByteGoHq/system-design-101 ★ 91k · visual explanations of each building block
  github.com/ByteByteGoHq/system-design-101
- theanalyst/awesome-distributed-systems ★ 12k · curated papers and courses
  github.com/theanalyst/awesome-distributed-systems

TRACK G

Low-level design (LLD)

Common round: design the classes for a system

Know

- OOP principles and SOLID, with examples you can write on the spot.
- Key design patterns: Factory, Strategy, Observer, Singleton, Adapter, Decorator, Repository.
- Composition over inheritance; designing with interfaces in TypeScript.

Practice problems

Parking lot, LRU cache, Splitwise, elevator system, movie ticket booking (the LLD version of TicketRush), vending machine, logger, in-memory rate limiter.

Resources · best first

- **START HERE** ashishps1/awesome-low-level-design ★ 27k · LLD concepts and interview problems with solutions
  github.com/ashishps1/awesome-low-level-design
- Refactoring.Guru: design patterns · free, beautifully illustrated
  refactoring.guru/design-patterns
- RefactoringGuru/design-patterns-typescript ★ 1.5k · the same patterns in TypeScript
  github.com/RefactoringGuru/design-patterns-typescript
- prasadgujar/low-level-design-primer ★ 8.0k · more LLD practice
  github.com/prasadgujar/low-level-design-primer
- Head First Design Patterns (book) · Freeman and Robson; the friendliest intro
  oreilly.com/library/view/head-first-design/9781492077992

TRACK H

High-level system design interview

Usually decides senior vs mid level

A framework for the 45–60 minute round

1. Clarify functional and non-functional requirements (scale, latency, consistency).
2. Back-of-envelope estimates: requests per second, storage, bandwidth.
3. API design: the main endpoints.
4. Data model and database choice, with reasons.
5. High-level diagram: clients, load balancer, services, cache, queue, databases.
6. Deep dive into one or two components the interviewer cares about.
7. Bottlenecks, failure modes, and trade-offs. Say out loud what you would do differently at 10x scale.

Classic questions to practice

URL shortener, rate limiter, notification system, chat app (WhatsApp), news feed, ticket booking, file storage (Dropbox), video streaming (YouTube), ride sharing (Uber), web crawler, search autocomplete, leaderboard, payment system, distributed key-value store.

Resources · best first

- **START HERE** donnemartin/system-design-primer ★ 375k · includes worked interview questions
  github.com/donnemartin/system-design-primer
- ashishps1/awesome-system-design-resources ★ 43k · free concepts and interview problems
  github.com/ashishps1/awesome-system-design-resources
- Alex Xu: System Design Interview Vol. 1 and 2 (ByteByteGo) · the most-used system design interview books
  bytebytego.com
- Hello Interview · free system design guides
  hellointerview.com
- Excalidraw · practice drawing your designs
  excalidraw.com

TRACK I

Data structures and algorithms (DSA)

Still common at product companies

Honest note

Many product companies still run DSA rounds even for experienced engineers; many startups and service companies weigh it less. Don’t pause the roadmap for it, but don’t skip it either. Run it in parallel at 3–4 problems a week.

Focus on patterns, not problem count

Arrays and hashing, two pointers, sliding window, stack, binary search, linked lists, trees, heaps, graphs (BFS/DFS), intervals, and basic dynamic programming.

Resources · best first

- **START HERE** yangshun/tech-interview-handbook ★ 143k · study plan, patterns, and interview tips
  github.com/yangshun/tech-interview-handbook
- NeetCode · NeetCode 150 list with video explanations
  neetcode.io
- seanprashad/leetcode-patterns ★ 14k · problems grouped by pattern
  github.com/seanprashad/leetcode-patterns
- LeetCode · where you’ll practice
  leetcode.com
- jwasham/coding-interview-university ★ 363k · optional: a full CS study plan if you want depth
  github.com/jwasham/coding-interview-university

TRACK J

Networking and the web

Easy points if you prepare

Know

- “What happens when you type a URL and press Enter?”, end to end, in detail.
- HTTP/1.1 vs HTTP/2 vs HTTP/3; keep-alive and connection reuse.
- Caching headers, ETags, and CDNs.
- Layer 4 vs layer 7 load balancers.

Resources · best first

- **START HERE** alex/what-happens-when ★ 43k · the classic interview question, answered in depth
  github.com/alex/what-happens-when
- vasanthk/how-web-works ★ 17k · a shorter version of the same journey
  github.com/vasanthk/how-web-works
- High Performance Browser Networking (free book) · Ilya Grigorik; TCP, TLS, HTTP/2
  hpbn.co

TRACK K
Cloud and infrastructure basics

Vocabulary, not mastery

Know

- Core AWS services at the concept level: EC2, S3, RDS, SQS/SNS, Lambda, CloudFront.
- Pre-signed URLs for direct browser-to-S3 file uploads.
- Kubernetes concepts: pods, deployments, services, autoscaling.
- Serverless trade-offs (cold starts, limits), and what infrastructure-as-code (Terraform) is for.

Resources · best first

- **START HERE** AWS Skill Builder · free “Cloud Practitioner Essentials” course
  skillbuilder.aws
- Kubernetes Basics tutorial · official interactive tutorial
  kubernetes.io/docs/tutorials/kubernetes-basics
- MichaelCade/90DaysOfDevOps ★ 30k · community DevOps learning journey
  github.com/MichaelCade/90DaysOfDevOps
- techiescamp/kubernetes-learning-path ★ 7.7k · structured Kubernetes path
  github.com/techiescamp/kubernetes-learning-path

TRACK L
Keep your Angular sharp

Your differentiator

Why

Full-stack interviews probe both halves of what you claim. Three years of Angular plus real backend depth is a rare and strong combination, so don’t let the frontend rust.

Know

- RxJS flattening operators: switchMap vs mergeMap vs concatMap vs exhaustMap, and when each fits.
- Change detection, OnPush, and signals.
- Lazy loading, bundle size, and rendering performance.
- State management trade-offs.

Resources · best first

- **START HERE** Angular docs · signals, change detection, and performance guides
  angular.dev
- Learn RxJS · operators explained with examples
  learnrxjs.io
- sudheerj/angular-interview-questions ★ 5.0k · quick revision
  github.com/sudheerj/angular-interview-questions
- PatrickJS/awesome-angular ★ 10k · curated Angular resources
  github.com/PatrickJS/awesome-angular

TRACK M

Behavioral round

Often the tie-breaker

Know

Answer with the STAR method: Situation, Task, Action, Result. Prepare 6–8 real stories: a production bug you fixed, a disagreement with a teammate, a trade-off decision, something you learned fast (this roadmap), a time you took ownership, and a mistake you made and what changed afterwards.

Tip

Your TicketRush decision log is a goldmine here: every entry is a ready-made “tell me about a technical decision you made” story.

Resources · best first

- **START HERE** ashishps1/awesome-behavioral-interviews ★ 9.0k · tips, common questions, and example answers
  github.com/ashishps1/awesome-behavioral-interviews
- Tech Interview Handbook: behavioral · how to structure STAR answers
  techinterviewhandbook.org/behavioral-interview

Self-test: can you explain these out loud?

Tick a box only when you can explain the answer clearly in two minutes without notes. These come up again and again in backend and full-stack interviews.

Node.js and JavaScript

- ☐ Walk through the event loop. What runs first: `process.nextTick`, `Promise.then`, `setTimeout(0)`, or `setImmediate`?
- ☐ Why is Node good for I/O-heavy work and bad for CPU-heavy work? How do you handle CPU work?
- ☐ What are streams and backpressure? When would you stream a file instead of reading it?
- ☐ How do you use all CPU cores in Node?
- ☐ How would you find a memory leak in a Node app?

Databases

- ☐ SQL vs NoSQL: how do you choose for a given feature?
- ☐ How does an index speed up reads, and what does it cost on writes?
- ☐ Explain ACID and isolation levels. What are dirty reads and phantom reads?
- ☐ Optimistic vs pessimistic locking: when would you use each?
- ☐ Replication vs sharding. What is replication lag and how can it affect users?
- ☐ What is the N+1 problem and how do you fix it?

APIs, auth, and security

- ☐ JWT vs sessions: trade-offs, and where you’d store tokens in an Angular app.
- ☐ Explain the OAuth 2.0 authorization code flow with PKCE.
- ☐ What is idempotency, and how do you make a payment API idempotent?
- ☐ Offset vs cursor pagination.
- ☐ How does CORS actually work, including preflight requests?
- ☐ What is IDOR and how do you prevent it?

Scaling and distributed systems

- ☐ What happens when you type a URL into the browser and press Enter?
- ☐ How do you scale WebSockets across multiple servers?
- ☐ Cache invalidation strategies. What is a cache stampede and how do you prevent it?
- ☐ Queue vs pub/sub vs stream. At-least-once vs exactly-once delivery.
- ☐ Explain the CAP theorem in plain words with an example.
- ☐ What problem does consistent hashing solve?
- ☐ Token bucket vs sliding window rate limiting.
- ☐ Monolith vs microservices: when would you split, and what does it cost?
- ☐ How do you make sure two people can't book the same seat? (You'll have built this.)

Suggested weekly rhythm

Activity	How much	Start from
Build the current roadmap phase	6–7 hours	Week 1
DSA problems (pattern by pattern)	3–4 problems, about 2–3 hours	Week 1
Node internals and JS/TS depth (Tracks A and B)	1 hour	Weeks 1–8
One low-level design problem	Every 2 weeks	Phase 2
One system design question on paper	1 per week	Phase 3
Explain one self-test question out loud	1 per week	Week 1
Mock interview with a friend or peer	Every 2 weeks	Month 4

If you only have 10 hours some weeks, cut DSA down to 2 problems before cutting build time. The build is what makes everything else stick.

Mentor rules

1. **Spend 30% of your time learning and 70% building.** If you've watched three videos without writing code, stop and build.
2. **Measure everything.** "It felt faster" means nothing; "p95 dropped from 420ms to 35ms" means everything.
3. **Keep a decision log in the repo.** For each choice, write one short note: what you chose, the alternatives, and why. This *is* architecture work, and it's gold in interviews.
4. **Don't start with Kubernetes, Kafka, or microservices.** They're the last 10%, and without the fundamentals they're just buzzwords.
5. **Practice explaining out loud.** Knowing something and explaining it under pressure are different skills. Record yourself answering one question a week and listen back.

Where this gets you

At the end you'll have a portfolio project that proves you can reason about a full system: an Angular frontend, an Express backend plus a NestJS service, load-test results, Grafana screenshots, tests, and a decision log. Combined with the interview track, that is exactly what moves people toward senior, full-stack, and tech-lead roles.

Your first step this week: read the MDN HTTP overview, install Postgres, create the TicketRush repo with the Express folder structure from Phase 1, and solve your first two DSA problems.

Note on resources: every GitHub repo and star count was checked directly on GitHub on 18 October 2026 (two repos had moved, and the links point to their new homes). Websites, YouTube channels, and books are well-known official sources, but they couldn't be opened live from this chat, so if one ever fails to load, search its name.

Appendix: every GitHub repo in this roadmap, ranked by stars

Stars checked on github.com on 18 October 2026. More stars means more developers have bookmarked and recommended it; it doesn't mean you need to read all of it. Use the "Where" column to see when each one becomes relevant.

#	Repository	Stars	Where
1	donnemartin/system-design-primer the most-starred system design resource on GitHub	★ 375k	Phase 9, Track H
2	nilbuild/developer-roadmap the project behind roadmap.sh (moved from kamranahmedse/developer-roadmap)	★ 370k	Phase 0
3	jwasham/coding-interview-university optional: a full CS study plan if you want depth	★ 363k	Track I
4	getify/You-Dont-Know-JS free book series on JS in depth	★ 185k	Track B
5	yangshun/tech-interview-handbook study plan, patterns, and interview tips	★ 143k	Track I
6	goldbergyoni/nodebestpractices sections 1 (project architecture) and 2 (error handling)	★ 106k	Phase 1
7	ByteByteGoHq/system-design-101 complex systems explained with visuals	★ 91k	Phase 9, Track F
8	binhnguyennus/awesome-scalability real engineering blogs from large companies	★ 75k	Phase 9
9	karanpratapsingh/system-design a free, complete system design course	★ 47k	Phase 9
10	alex/what-happens-when the classic interview question, answered in depth	★ 43k	Track J
11	ashishps1/awesome-system-design-resources free concepts and interview problems	★ 43k	Track H
12	OWASP/CheatSheetSeries source of the OWASP Cheat Sheet Series	★ 34k	Phase 6
13	grafana/k6 the k6 load-testing tool	★ 32k	Phase 8
14	MichaelCade/90DaysOfDevOps community DevOps learning journey	★ 30k	Track K
15	sudheerj/javascript-interview-questions quick revision before interviews	★ 28k	Track B
16	ashishps1/awesome-low-level-design LLD concepts and interview problems with solutions	★ 27k	Track G
17	goldbergyoni/javascript-testing-best-practices the reference for JS/Node testing	★ 25k	Track E
18	microsoft/api-guidelines REST API design guidelines	★ 23k	Track C
19	milanm/DevOps-Roadmap optional: where DevOps goes next	★ 21k	Phase 7

#	Repository	Stars	Where
20	vasanthk/how-web-works what happens behind the scenes of a web request	★ 17k	Phase 0, Track J
21	Sairyss/domain-driven-hexagon architecture patterns with TypeScript code	★ 15k	Phase 9
22	forwardemail/supertest HTTP API testing	★ 14k	Track E
23	seanprashad/leetcode-patterns problems grouped by pattern	★ 14k	Track I
24	juice-shop/juice-shop deliberately vulnerable Angular + Node app to hack	★ 14k	Phase 6
25	theanalyst/awesome-distributed-systems curated papers and courses	★ 12k	Track F
26	pingcap/awesome-database-learning go deeper into database internals	★ 11k	Phase 2
27	PatrickJS/awesome-angular curated Angular resources	★ 10k	Track L
28	ashishps1/awesome-behavioral-interviews tips, common questions, and example answers	★ 9.0k	Track M
29	prasadgujar/low-level-design-primer more LLD practice	★ 8.0k	Track G
30	techiescamp/kubernetes-learning-path structured Kubernetes path	★ 7.7k	Track K
31	rabbitmq/rabbitmq-tutorials runnable code for those tutorials	★ 6.9k	Phase 5
32	prakhar1989/docker-curriculum hands-on Docker tutorial	★ 6.1k	Phase 7
33	sudheerj/angular-interview-questions quick revision	★ 5.0k	Track L
34	goldbergyoni/nodejs-testing-best-practices beyond the basics: integration and component tests	★ 4.4k	Track E
35	testcontainers/testcontainers-node tests against a real Postgres or Redis	★ 2.6k	Track E
36	Sairyss/backend-best-practices backend practices with TypeScript examples	★ 2.4k	Phase 1
37	RefactoringGuru/design-patterns-typescript the same patterns in TypeScript	★ 1.5k	Track G